19.1 通道适配器设计模式

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~560k tokens，输出 ~50k tokens（本章合计）

OpenClaw 的核心定位是"多通道 AI Agent 网关"——同一个 AI Agent 可以同时出现在 Telegram、WhatsApp、Discord、Slack、Signal 等数十个消息平台上。这意味着系统必须将每个平台的独特 API 转换为统一的内部格式。实现这一目标的关键架构就是**通道适配器模式（Channel Adapter Pattern）**。

> **衍生解释：适配器模式（Adapter Pattern）**
>
> 适配器模式是 GoF（Gang of Four）经典设计模式之一，属于结构型模式。它通过一个"适配器"对象，将一个类的接口转换为客户端期望的另一种接口，使原本因接口不兼容而无法协作的类能够一起工作。在 OpenClaw 中，每个通道插件就是一个适配器——它将 Telegram Bot API、WhatsApp Web 协议、Discord.js 等各不相同的接口，转换为 Gateway 能够统一处理的 `ChannelPlugin` 接口。

19.1.1 通道核心接口分析

ChannelPlugin：统一的通道契约

每个通道在 OpenClaw 中的完整定义由 `ChannelPlugin` 类型描述。这是一个包含 20+ 个可选适配器的巨型接口，定义在 `src/channels/plugins/types.plugin.ts`：

```typescript
// src/channels/plugins/types.plugin.ts（简化）
export type ChannelPlugin<ResolvedAccount = any, Probe = unknown, Audit = unknown> = {
  id: ChannelId;                          // 通道标识符，如 "telegram"
  meta: ChannelMeta;                      // 显示元数据（名称、图标、文档链接等）
  capabilities: ChannelCapabilities;      // 能力声明

  // ── 配置与设置 ──
  config: ChannelConfigAdapter<ResolvedAccount>;   // 账号配置管理（必需）
  configSchema?: ChannelConfigSchema;              // JSON Schema 配置校验
  setup?: ChannelSetupAdapter;                     // CLI 初始化向导
  onboarding?: ChannelOnboardingAdapter;           // 引导流程

  // ── 安全与权限 ──
  security?: ChannelSecurityAdapter<ResolvedAccount>;
  pairing?: ChannelPairingAdapter;
  auth?: ChannelAuthAdapter;
  elevated?: ChannelElevatedAdapter;

  // ── 消息处理 ──
  outbound?: ChannelOutboundAdapter;       // 出站消息发送
  messaging?: ChannelMessagingAdapter;     // 消息工具目标解析
  streaming?: ChannelStreamingAdapter;     // 流式传输参数
  threading?: ChannelThreadingAdapter;     // 线程/引用回复
  mentions?: ChannelMentionAdapter;        // @提及处理
  actions?: ChannelMessageActionAdapter;   // 消息动作（反应、编辑、撤回）

  // ── 运行时 ──
  gateway?: ChannelGatewayAdapter<ResolvedAccount>;  // Gateway 服务生命周期
  commands?: ChannelCommandAdapter;                  // 命令权限
  groups?: ChannelGroupAdapter;                      // 群组行为
  heartbeat?: ChannelHeartbeatAdapter;               // 心跳检查
  status?: ChannelStatusAdapter<...>;                // 状态探测与审计

  // ── 扩展 ──
  agentPrompt?: ChannelAgentPromptAdapter;  // 注入通道特有的系统提示词
  directory?: ChannelDirectoryAdapter;       // 联系人/群组目录
  resolver?: ChannelResolverAdapter;         // 目标解析
  agentTools?: ChannelAgentToolFactory | ChannelAgentTool[];  // 通道专属工具
};
```

这个接口体现了**接口分离原则（Interface Segregation Principle, ISP）**——每个适配器都是一个独立的可选字段。一个最简通道只需要实现 `id` 、 `meta` 、 `capabilities` 和 `config` 四个必需项；其他所有适配器都可以按需添加。

ChannelCapabilities：能力声明

`ChannelCapabilities` 声明了一个通道支持哪些功能，让系统在运行时按需启用或禁用特性：

```
// src/channels/plugins/types.core.ts
export type ChannelCapabilities = {
  chatTypes: Array<NormalizedChatType | "thread">;  // 支持的对话类型
  polls?: boolean;           // 投票
  reactions?: boolean;       // 表情反应
  edit?: boolean;            // 消息编辑
  unsend?: boolean;          // 撤回消息
  reply?: boolean;           // 引用回复
  effects?: boolean;         // 消息特效
  groupManagement?: boolean;  // 群组管理
  threads?: boolean;         // 线程
  media?: boolean;           // 媒体附件
  nativeCommands?: boolean; // 原生斜杠命令
  blockStreaming?: boolean; // 流式分块回复
};
```

不同通道的能力差异很大：

通道	chatTypes	polls	reactions	threads	nativeCommands	blockStreaming
Telegram	direct, group, channel, thread	❌	❌	❌	✅	✅
WhatsApp	direct, group	✅	✅	❌	❌	❌
Discord	direct, channel, thread	✅	✅	✅	✅	❌
Slack	direct, channel, thread	❌	✅	✅	✅	❌
Signal	direct, group	❌	✅	❌	❌	❌

这些声明不只是文档——系统会在运行时检查它们。比如，只有 `blockStreaming: true` 的通道才会启用流式分块回复（第 10 章）。

ChannelMeta：显示元数据

每个通道都有一组描述性元数据，用于 CLI 界面、Web UI 和文档链接：

```
// src/channels/plugins/types.core.ts（简化）
export type ChannelMeta = {
  id: ChannelId;
  label: string;              // 显示名，如 "Telegram"
  selectionLabel: string;     // 选择界面标签，如 "Telegram (Bot API)"
  docsPath: string;           // 文档路径，如 "/channels/telegram"
  blurb: string;              // 一句话描述
  order?: number;             // 排序权重
  aliases?: string[];         // 别名（如 "imsg" → "imessage"）
  systemImage?: string;       // SF Symbols 图标名
  // ...更多 UI 相关字段
};
```

核心通道的元数据硬编码在 `src/channels/registry.ts` 中：

```
// src/channels/registry.ts（摘录）
const CHAT_CHANNEL_META: Record<ChatChannelId, ChannelMeta> = {
  telegram: {
    id: "telegram",
    label: "Telegram",
    selectionLabel: "Telegram (Bot API)",
    blurb: "simplest way to get started — register a bot with @BotFather and get going.",
    systemImage: "paperplane",
    // ...
  },
  whatsapp: {
    id: "whatsapp",
    label: "WhatsApp",
    selectionLabel: "WhatsApp (QR link)",
    blurb: "works with your own number; recommend a separate phone + eSIM.",
    // ...
  },
  // ...其他通道
};
```

19.1.2 通道配置类型系统

联邦式配置

OpenClaw 的通道配置采用**联邦式（Federated）**结构——每个通道的配置类型独立定义，然后在顶层 `ChannelsConfig` 中组合：

```
// src/config/types.channels.ts
export type ChannelsConfig = {
  defaults?: ChannelDefaultsConfig;
  whatsapp?: WhatsAppConfig;
  telegram?: TelegramConfig;
  discord?: DiscordConfig;
  googlechat?: GoogleChatConfig;
  slack?: SlackConfig;
  signal?: SignalConfig;
  imessage?: IMessageConfig;
  msteams?: MSTeamsConfig;
  [key: string]: unknown;       // 扩展通道的动态配置
};
```

注意最后的索引签名 `[key: string]: unknown` ——这允许扩展通道（如 Matrix、Twitch、LINE 等）在不修改核心类型定义的情况下添加自己的配置。

每个通道的配置类型由各自的类型文件定义（如 `types.telegram.ts` 、 `types.whatsapp.ts` 等），并通过通道插件的 `config` 适配器进行读取和管理。

ChannelConfigAdapter：配置访问层

`ChannelConfigAdapter` 是每个通道**必须实现**的适配器，提供账号列表、账号解析、启用/禁用等基本操作：

```typescript
// src/channels/plugins/types.adapters.ts（简化）
export type ChannelConfigAdapter<ResolvedAccount> = {
  listAccountIds: (cfg: OpenClawConfig) => string[];       // 列出所有账号 ID
  resolveAccount: (cfg: OpenClawConfig, accountId?) => ResolvedAccount;  // 解析账号
  defaultAccountId?: (cfg: OpenClawConfig) => string;      // 默认账号
  setAccountEnabled?: (...) => OpenClawConfig;             // 启用/禁用
  deleteAccount?: (...) => OpenClawConfig;                 // 删除账号
  isEnabled?: (account, cfg) => boolean;                   // 是否启用
  isConfigured?: (account, cfg) => boolean | Promise<boolean>;  // 是否已配置
  describeAccount?: (account, cfg) => ChannelAccountSnapshot;   // 账号快照
  resolveAllowFrom?: (...) => string[] | undefined;        // 白名单
  formatAllowFrom?: (...) => string[];                     // 白名单格式化
};
```

`ResolvedAccount` 是一个泛型参数——不同通道的"解析后账号"结构完全不同。Telegram 的解析账号包含 `token`、`allowFrom`、`replyToMode` 等字段；WhatsApp 的则包含 QR 码登录状态、重连参数等。通过泛型，系统在保持类型安全的同时允许各通道自由定义内部结构。

通道默认配置

`ChannelDefaultsConfig` 提供跨通道的全局默认值：

```typescript
// src/config/types.channels.ts
export type ChannelDefaultsConfig = {
  groupPolicy?: GroupPolicy;
  heartbeat?: ChannelHeartbeatVisibilityConfig;
};

export type ChannelHeartbeatVisibilityConfig = {
  showOk?: boolean;          // 是否显示正常心跳（默认 false）
  showAlerts?: boolean;      // 是否显示告警（默认 true）
  useIndicator?: boolean;    // 是否使用 UI 状态指示器（默认 true）
};
```

这允许用户在一个地方设定所有通道的默认行为，各通道可以覆盖这些默认值。

19.1.3 通道与 Gateway 的连接桥

ChannelDock：轻量级连接层

在 `ChannelPlugin` 和 Gateway 之间，存在一个名为 `ChannelDock` （通道船坞）的轻量级中间层，定义在 `src/channels/dock.ts` ：

```
// src/channels/dock.ts（简化）
export type ChannelDock = {
  id: ChannelId;
  capabilities: ChannelCapabilities;
  commands?: ChannelCommandAdapter;
  outbound?: { textChunkLimit?: number };
  streaming?: ChannelDockStreaming;
  elevated?: ChannelElevatedAdapter;
  config?: { resolveAllowFrom?; formatAllowFrom? };
  groups?: ChannelGroupAdapter;
  mentions?: ChannelMentionAdapter;
  threading?: ChannelThreadingAdapter;
  agentPrompt?: ChannelAgentPromptAdapter;
};
```

为什么需要这个中间层？因为 `ChannelPlugin` 是"重量级"的——它可能导入监控器、Web 登录流、puppeteer 等大型依赖。而 Gateway 的许多共享代码路径（回复处理、命令鉴权、沙盒解释等）只需要通道的一小部分轻量信息。

```
┌──────────────────────┐
│   Gateway 共享代码      │
│  (reply, commands,     │
│   auth, sandbox...)    │
└──────────────────────┘
           │
           │ 轻量访问
           ▼
┌──────────────────────┐
│     ChannelDock        │  ← 只包含元数据 + 轻量适配器
│   (dock.ts)            │
└──────────────────────┘
           │
           │ 仅在需要时
           ▼
┌──────────────────────┐
│     ChannelPlugin      │  ← 完整通道实现（重量级）
│  (plugins/*.ts)        │
└──────────────────────┘
```

源码中的注释明确了这一设计意图：

```
// src/channels/dock.ts
// Channel docks: lightweight channel metadata/behavior for shared code paths.
//
// Rules:
// - keep this module *light* (no monitors, probes, puppeteer/web login, etc)
// - OK: config readers, allowFrom formatting, mention stripping patterns
// - shared code should import from here, not from the plugins registry
```

核心通道的 Dock 注册

七个核心通道的 dock 硬编码在 `DOCKS` 对象中：

```
// src/channels/dock.ts（摘录）
const DOCKS: Record<ChatChannelId, ChannelDock> = {
  telegram: {
    id: "telegram",
    capabilities: {
      chatTypes: ["direct", "group", "channel", "thread"],
      nativeCommands: true,
      blockStreaming: true,
    },
    outbound: { textChunkLimit: 4000 },
    groups: { resolveRequireMention: resolveTelegramGroupRequireMention, ... },
    threading: {
      resolveReplyToMode: ({ cfg }) => cfg.channels?.telegram?.replyToMode ?? "first",
      buildToolContext: ({ context, hasRepliedRef }) => ({ ... }),
    },
  },
  discord: {
    id: "discord",
    capabilities: {
      chatTypes: ["direct", "channel", "thread"],
      polls: true, reactions: true, threads: true, nativeCommands: true,
    },
    outbound: { textChunkLimit: 2000 },           // Discord 2000 字符限制
    streaming: {
      blockStreamingCoalesceDefaults: { minChars: 1500, idleMs: 1000 },
    },
    // ...
  },
  // ...其他通道
};
```

每个通道的 `textChunkLimit` 不同——Telegram 和 WhatsApp 是 4000 字符，Discord 只有 2000 字符。这些差异直接影响出站消息的分块策略。

通道注册表

所有通道（核心 + 扩展）通过 `registry.ts` 统一管理：

```
// src/channels/registry.ts
export const CHAT_CHANNEL_ORDER = [
  "telegram", "whatsapp", "discord", "googlechat",
  "slack", "signal", "imessage",
] as const;

export type ChatChannelId = (typeof CHAT_CHANNEL_ORDER)[number];

// 别名映射
export const CHAT_CHANNEL_ALIASES: Record<string, ChatChannelId> = {
  imsg: "imessage",
  "google-chat": "googlechat",
  gchat: "googlechat",
};
```

通道 ID 的归一化函数 `normalizeChannelId` 支持大小写不敏感匹配和别名解析：

```
// src/channels/registry.ts
export function normalizeChatChannelId(raw?: string | null): ChatChannelId | null {
  const normalized = raw?.trim().toLowerCase();
  if (!normalized) return null;
  const resolved = CHAT_CHANNEL_ALIASES[normalized] ?? normalized;
  return CHAT_CHANNEL_ORDER.includes(resolved) ? resolved : null;
}
```

对于扩展通道（非核心七大通道），`normalizeAnyChannelId` 会查询插件注册表：

```
// src/channels/registry.ts
export function normalizeAnyChannelId(raw?: string | null): ChannelId | null {
  const key = raw?.trim().toLowerCase();
  if (!key) return null;

  const registry = requireActivePluginRegistry();
  const hit = registry.channels.find((entry) => {
    const id = String(entry.plugin.id).trim().toLowerCase();
    if (id === key) return true;
    // 检查别名
    return (entry.plugin.meta.aliases ?? []).some(alias => alias.trim().toLowerCase() === k
  });
  return hit?.plugin.id ?? null;
}
```

插件到 Dock 的桥接

扩展通道的 `ChannelPlugin` 可以自动转换为 `ChannelDock`：

```
// src/channels/dock.ts
function buildDockFromPlugin(plugin: ChannelPlugin): ChannelDock {
  return {
    id: plugin.id,
    capabilities: plugin.capabilities,
    commands: plugin.commands,
    outbound: plugin.outbound?.textChunkLimit
      ? { textChunkLimit: plugin.outbound.textChunkLimit }
      : undefined,
    streaming: plugin.streaming
      ? { blockStreamingCoalesceDefaults: plugin.streaming.blockStreamingCoalesceDefaults }
      : undefined,
    elevated: plugin.elevated,
    config: plugin.config ? { resolveAllowFrom: ..., formatAllowFrom: ... } : undefined,
    groups: plugin.groups,
    mentions: plugin.mentions,
    threading: plugin.threading,
    agentPrompt: plugin.agentPrompt,
  };
}
```

这个函数提取 `ChannelPlugin` 中的轻量部分，构建出不依赖重量级导入的 `ChannelDock` 。扩展通道的开发者无需手动维护两份定义——只需实现 `ChannelPlugin` ，系统会自动生成对应的 dock。

通道解析的完整流程

从用户消息到找到对应通道的完整路径：

```
入站消息 (Channel="telegram")
    │
    ▼
normalizeChannelId("telegram")
    │ 查核心注册表
    ▼
ChatChannelId = "telegram"
    │
    ├── getChannelDock("telegram")  → ChannelDock（轻量，用于共享代码）
    │
    └── getChannelPlugin("telegram") → ChannelPlugin（完整，用于通道特有操作）
```

这个分层设计确保了：

- 共享代码路径保持轻量和快速
- 通道特有逻辑被隔离在插件中
- 新通道可以通过插件系统零侵入地接入

本节小结

1. **`ChannelPlugin` 接口**是通道适配器的核心契约，包含 20+ 个可选适配器字段，遵循接口分离原则——只需实现必要的 `id` 、 `meta` 、 `capabilities` 和 `config` 即可。
2. **`ChannelCapabilities`** 通过声明式的能力标记（polls、reactions、threads 等），使系统能够在运行时适应性地启用特性。
3. **联邦式配置类型系统**让每个通道独立定义配置结构，通过索引签名 `[key: string]: unknown` 支持扩展通道的动态配置。
4. **ChannelDock 中间层**将轻量元数据从重量级插件中分离，确保共享代码路径不会因导入通道实现而引入不必要的依赖。
5. **通道注册表**通过 `CHAT_CHANNEL_ORDER` 、别名映射和插件注册表三层查找，实现了灵活的通道 ID 归一化。
6. 整个架构体现了 OpenClaw 的核心设计哲学：**统一接口、差异实现、按需加载**。